

UNIT I

1. Software Crisis: What to Add

Y2K as a software crisis

- **The cause:** to save scarce memory and storage, many old programs stored the year as two digits (99, not 1999). On 1 January 2000, '00' could be read as 1900.
- **The risk:** wrong date arithmetic, failed interest and billing calculations, expired-record deletion, and possible failures in banking, power, telecom and embedded control systems.
- **The response:** a worldwide remediation effort: inventory the code, find date fields, fix (expand to four digits, or use windowing), test and re-deploy. Estimates of global spend run into hundreds of billions of dollars.
- **Why it counts as a crisis:** it came from short-sighted design and years of neglected maintenance on legacy code. Documentation was missing, original programmers had left, and the deadline could not move.
- **Counter-view:** it was largely averted, few serious failures occurred, and it was a single known deadline, unlike the continuing failures that define the software crisis.

Conclusion: Y2K was a costly but manageable symptom of the crisis. It showed the price of poor design decisions and weak maintenance, and it was resolved through planning and disciplined engineering.

Why the crisis is 'perpetual'

- 1 **Growing size and complexity.** Systems grow faster than our ability to understand and control them.
- 2 **Changing requirements.** Customers change their minds while the software is being built.
- 3 **Technology outruns practice.** New platforms and tools arrive before the old practices mature.
- 4 **Maintenance burden of legacy systems.** Most effort goes into changing old, poorly documented code.
- 5 **Shortage of skilled staff and weak project management.** Estimates, schedules and quality control are often poor.
- 6 **Brooks' 'No Silver Bullet'.** Essential complexity (the difficulty of the problem itself) cannot be removed by any single tool or method, so improvement is gradual, never final.

2. What Is Software Engineering?

Definitions

- **IEEE:** the application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software; that is, the application of engineering to software.
- **Fritz Bauer (1968 NATO conference):** the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.

Art, craft or science?

- **Art:** creativity in architecture and design, and in finding elegant solutions.
- **Craft:** early practice was experience-based and ad hoc, learned by apprenticeship.
- **Science:** measurable principles exist, such as Halstead's metrics, COCOMO and other estimation models.
- **Conclusion:** software engineering is an engineering discipline that blends all three: creative design, practical skill, and measurable scientific principles.

Software engineering versus traditional engineering

Aspect	Traditional engineering	Software engineering
Product	Physical, tangible	Intangible, logical
Production	Manufactured; has a production phase	Developed; copying is trivial, so no production phase
Ageing	Parts wear out	Does not wear out but deteriorates through change
Where cost lies	Largely in production and materials	Largely in design and development
Change	Costly and slow	Easy to make but risky

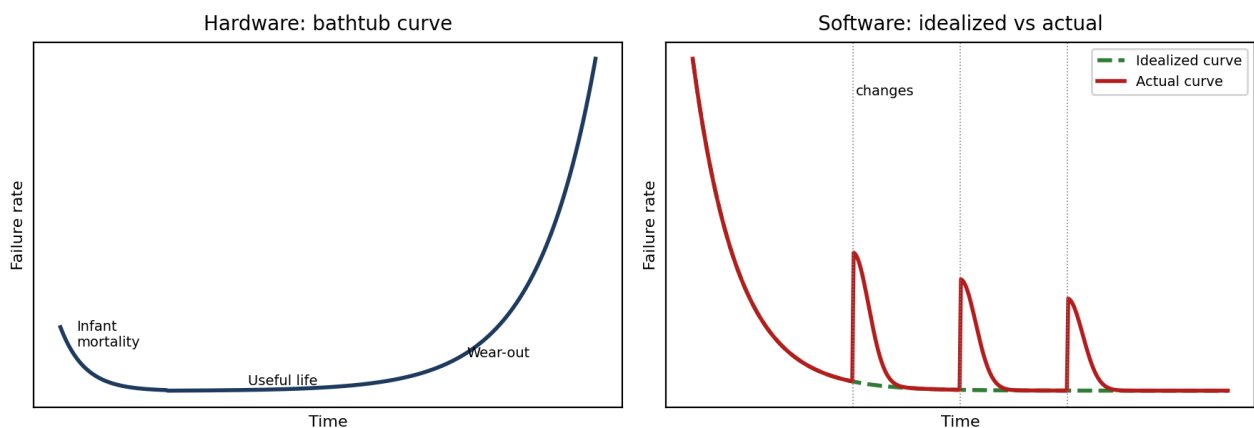
Aspect	Traditional engineering	Software engineering
Components	Many standard, reusable parts	Fewer standard components; much is custom-built
Standards and laws	Mature, well established	Still maturing
Requirements	Relatively stable	More volatile
Progress	Visible physically	Harder to see and measure

3. Characteristics of Software

- **Engineered, not manufactured.** The cost is in design; copies cost almost nothing, so quality is built in during development.
- **Does not wear out.** There is no physical ageing (see the curves below).
- **Mostly custom-built.** Most software is written for a specific need rather than assembled from off-the-shelf components, though reuse is growing.
- **Intangible and complex.** It cannot be touched, and its behaviour depends on many interacting states.
- **Easily modifiable,** which is both an advantage and a source of new defects.

Failure-rate curves

- **Hardware (bathtub curve):** high failure rate early (infant mortality from manufacturing defects), then a low, flat useful-life period, then a rising rate at the end as parts wear out.
- **Software, idealized curve:** high failure rate early as undiscovered defects surface and are fixed, then a low, flat curve. There is no wear-out rise.
- **Software, actual curve:** each change or upgrade introduces new defects, producing a spike. Each spike settles, but never quite back to the earlier minimum. Over time the baseline creeps up: the software deteriorates.



4. SDLC Phases

Phase	What happens	Who	Output
Feasibility study (preliminary investigation)	Decide if the project is worth doing: technical, economic, operational and schedule feasibility; alternatives and rough cost-benefit.	Analysts, management	Feasibility report
Requirements analysis and specification	Gather, analyse and document what the system must do.	Analysts, customer	SRS
Design	Decide how: architecture, modules, data structures, interfaces.	Designers, architects	Design document (HLD and LLD)
Coding (implementation)	Translate design into code; unit testing.	Programmers	Source code
Testing	Integration, system and acceptance testing to find and fix defects.	Testers, QA	Test plan, test cases, test reports

Phase	What happens	Who	Output
Deployment and maintenance	Install and use the system; corrective, adaptive, perfective and preventive changes.	Operations and maintenance team	Deployed system, maintenance log

Purpose of an SDLC

- Gives all stakeholders a **common process** and vocabulary.
- Defines **clear milestones** and deliverables at the end of each phase.
- Provides **control**: progress can be tracked, reviewed and managed.

5. Waterfall and Spiral Phases

Waterfall model

- 1 **Requirements**: gather and document requirements (SRS).
- 2 **Design**: convert requirements into system and software design.
- 3 **Implementation**: write the code from the design.
- 4 **Testing**: verify the code against the requirements.
- 5 **Deployment**: release the product to the users.
- 6 **Maintenance**: fix defects and enhance after release.

Feedback (iterative waterfall): the strict waterfall has no way back once a phase is finished. The iterative waterfall adds feedback paths so that errors found in a later phase can be corrected in the earlier one, which is far more practical.

Spiral model

- **Dimensions**: the **radial** dimension shows cumulative cost; the **angular** dimension shows progress through the cycle.
- **Each loop is a phase** of the project, for example feasibility, requirements, design, and so on.
- **Each loop passes through four quadrants**: (1) determine objectives, alternatives and constraints; (2) identify and resolve risks (with prototypes and analysis); (3) develop and verify the next-level product; (4) plan the next loop with customer review.
- **Beneficial where** (use the PPT's 'Best Suited For' list): large, high-risk and complex projects; unclear or evolving requirements; new or unproven technology; projects needing frequent customer feedback and risk-driven planning.

6. Agile: Extra Depth for the 10-Mark Question

The 12 principles of Agile (brief)

- 1 Satisfy the customer through early and continuous delivery of valuable software.
- 2 Welcome changing requirements, even late in development.
- 3 Deliver working software frequently, from a couple of weeks to a couple of months.
- 4 Business people and developers work together daily.
- 5 Build projects around motivated individuals and trust them.
- 6 Face-to-face conversation is the most effective way to communicate.
- 7 Working software is the primary measure of progress.
- 8 Maintain a sustainable pace of development.
- 9 Continuous attention to technical excellence and good design.
- 10 Simplicity: maximise the amount of work not done.
- 11 Self-organising teams produce the best designs.
- 12 At regular intervals the team reflects and adjusts its behaviour.

Scrum in detail

Roles	Artifacts	Ceremonies (events)
Product Owner: owns the vision and the product backlog; sets priorities. Scrum Master: coaches the team, removes obstacles, ensures Scrum is followed. Development Team: cross-functional, self-organising; builds the increment.	Product backlog: ordered list of everything the product might need. Sprint backlog: items chosen for the sprint plus the plan to deliver them. Increment: the working, potentially releasable product at the end of the sprint.	Sprint planning: decide what to build and how in the coming sprint. Daily stand-up (15 min): what I did, what I will do, what blocks me. Sprint review: demonstrate the increment to stakeholders and get feedback. Sprint retrospective: the team inspects its process and plans improvements.

Extreme Programming (XP) practices

- **Pair programming:** two developers at one workstation; one writes, one reviews.
- **Test-driven development (TDD):** write a failing test first, then code to pass it.
- **Refactoring:** continuously improve the code structure without changing behaviour.
- **Continuous integration:** integrate and test the code several times a day.
- **Small releases:** ship small, working versions frequently.

User stories

Requirements are written as short user stories in the form '**As a <role>, I want <feature> so that <benefit>**', for example: 'As a member, I want to reserve a book so that I can borrow it when it is returned.' Stories are estimated, prioritised in the backlog, and detailed through conversation and acceptance criteria.

Wider comparison of models

Parameter	Waterfall	Prototype	Spiral	Agile
Documentation	Heavy	Moderate	Heavy	Light, working software preferred
Cost	Low for stable projects; costly if change	Moderate; prototype cost	High (risk analysis)	Moderate; controlled by iterations
Team size	Any, large well supported	Small to medium	Large, specialised	Small, co-located (often 5-9)
Testing	After coding, late	Each prototype	Each loop	Continuous, every sprint
Change handling	Poor	Good	Good	Excellent
Delivery time	Long, at the end	Working prototype early	Delivered incrementally by loop	Frequent, every 1-4 weeks
Phase overlap	None (sequential)	Some	Iterative loops	High: phases overlap within a sprint
Customer involvement	Low	High	Moderate to high	Very high, continuous

7. RAD Model

Rapid Application Development (RAD) is an incremental model that emphasises very short development cycles, component-based construction and heavy user involvement to deliver a fully functional system quickly.

Phases

- 1 **Business modeling:** model the flow of information between business functions.
- 2 **Data modeling:** refine the information into data objects and define their relationships.
- 3 **Process modeling:** define how data objects are transformed to support business functions; add, modify and delete processing descriptions.

- 4 **Application generation:** build the system using fourth-generation tools and reusable components.
- 5 **Testing and turnover:** test new components and interfaces; reused components are already tested, which shortens this phase.

Key ideas

- **Time-boxing:** a fixed, short delivery window, usually 60 to 90 days.
- **Component reuse** to cut coding effort.
- **JAD (Joint Application Development)** sessions bring users and developers together to fix requirements quickly.

Advantages	Disadvantages
Very fast delivery; reduced development time Encourages customer feedback Reuse lowers effort and defects Progress is visible early	Needs skilled developers and strong teams Only suits modular systems Needs highly committed users Not suited to high technical risk or heavy performance demands

Use when: the system is modular, the team is skilled, technical risk is low, and the business can commit users to the schedule.

RAD versus Prototype

Point	RAD	Prototype model
Aim	Deliver the complete working system fast	Clarify requirements through a model
Time	Fixed short time-box (60-90 days)	No fixed limit
Final product	Built from the RAD output itself	Prototype often discarded and rebuilt
Reuse and tools	Heavy reuse, 4GL tools	Rapid tools, reuse optional
Requirements	Well understood, modular	Vague or unclear

8. Requirement Elicitation Techniques

Definition: requirements elicitation is the process of discovering, gathering and understanding what stakeholders need from the system. **Three classic problems:** (1) problems of scope (unclear boundaries or too much detail); (2) problems of understanding (users unsure of needs, poor communication, ambiguity); (3) problems of volatility (requirements change over time).

Techniques

- **Interviews.** One-to-one or group talks with stakeholders. *Structured* interviews use a fixed question list, are consistent and easy to compare; *unstructured* interviews are open and exploratory. Pros: rich detail, clarification, builds rapport. Cons: time-consuming, depends on interviewer skill, interviewee bias, users may omit things they take for granted.
- **Questionnaires and surveys.** Written questions sent to many stakeholders. Pros: cheap, reaches many people, easy to analyse. Cons: low response rate, cannot probe or clarify, questions must be designed carefully.
- **Brainstorming.** A group generates many ideas quickly without criticism, then ideas are grouped and prioritised. Pros: creative and inclusive. Cons: can be dominated by strong voices and needs a good facilitator.
- **Observation (ethnography).** Watch users do their real work in their environment. Pros: reveals tacit knowledge and real workflow. Cons: slow, intrusive, and users may behave differently when watched.
- **Document and existing-system analysis.** Study forms, reports, manuals and the current system. Pros: fast, factual, a good starting point. Cons: documents can be outdated and miss unwritten practice.
- **Prototyping.** Build a quick mock-up so users can react to something concrete. Pros: uncovers missing or wrong requirements early. Cons: users may mistake it for the real system; cost of building it.
- **Use cases and scenarios.** Describe interactions between actors and system in stories and step sequences. Pros: user-centred, easy to understand. Cons: can miss non-functional needs.
- **JAD (Joint Application Development).** A structured workshop of users, analysts and developers with a facilitator and scribe; similar in spirit to FAST.

FAST and QFD (from the PPT, add to the long answer)

- **FAST (Facilitated Application Specification Technique):** joint team of customers and developers meets, prepares lists of objects, services, constraints and performance criteria, then builds a consensus mini-specification.
- **QFD (Quality Function Deployment):** translates customer needs into technical requirements, classified as *normal*, *expected* and *exciting* requirements.

When to use which

Situation	Best technique
Stakeholders few, need detail	Interviews
Many, geographically spread users	Questionnaires
Need new ideas, unclear problem	Brainstorming, JAD, FAST
Complex real workflow, tacit knowledge	Observation
Replacing or upgrading an existing system	Document and system analysis
Users cannot articulate needs	Prototyping
Need to define interactions and behaviour	Use cases and scenarios

9. Analysis Models: Worked Case Studies

9.1 Library Management System

Assumptions: a member can borrow a limited number of books; each title can have several copies; a librarian issues, returns and manages the catalogue; fines are charged for late returns.

Element	Items
Actors	Member, Librarian, Administrator (optional: Supplier)
Entities (ER)	BOOK (BookID, Title, Author, ISBN, Publisher, Copies), MEMBER (MemberID, Name, Address, Phone, Type), LOAN (LoanID, IssueDate, DueDate, ReturnDate), FINE, and optionally RESERVATION, STAFF
Relationships	MEMBER borrows BOOK (M:N, resolved through LOAN); MEMBER pays FINE (1:N); STAFF issues LOAN (1:N)
Use cases	Search catalogue, Borrow book, Return book, Reserve book, Pay fine, Manage members, Manage books, Generate reports
Processes (DFD)	1 Manage Members, 2 Manage Books, 3 Issue Book, 4 Return Book, 5 Calculate Fine, 6 Generate Reports
Data stores	D1 Members, D2 Books, D3 Loans, D4 Fines
External entities	Member, Librarian

- **ER diagram:** draw MEMBER and BOOK as entities with attributes (underline the keys), connect with 'borrows' (M:N) and show LOAN as the associative entity carrying issue and due dates.
- **Use case diagram:** Member is linked to Search, Borrow, Return, Reserve, Pay Fine. Librarian is linked to Issue, Return, Manage Books and Members. 'Borrow' includes 'Check availability' and 'Verify membership'; 'Pay fine' extends 'Return book' (only when late).
- **DFD Level 0 (context):** one process 'Library Management System' with Member and Librarian as entities: Book request, Return, Fine payment, Receipts and Reports.
- **DFD Level 1:** decompose into processes 1-6 above with stores D1-D4. **Level 2:** expand 'Issue Book' into 3.1 Validate member, 3.2 Check book availability, 3.3 Check borrow limit, 3.4 Record loan and update copies.

9.2 Hotel Management System

Assumptions: guests book rooms in advance or walk in; receptionist handles check-in, check-out and billing; rooms have types and rates.

Element	Items
Actors	Guest, Receptionist, Manager, (Housekeeping, Payment system)
Use cases	Check room availability, Make reservation, Cancel reservation, Check in, Check out, Generate bill, Make payment, Manage rooms and rates, View reports
Processes (Level 1)	1 Reservation, 2 Check-in, 3 Room and Service Management, 4 Billing and Payment, 5 Reporting
Data stores	D1 Guests, D2 Rooms, D3 Reservations, D4 Billing
Entities	Guest, Receptionist, Manager

- **Use case diagram:** Guest with Reservation, Cancel, Make payment; Receptionist with Check in, Check out, Generate bill; Manager with Manage rooms and Reports. 'Make reservation' includes 'Check availability'; 'Generate bill' includes 'Calculate charges'; 'Apply discount' extends 'Generate bill'.
- **Level-1 DFD:** Guest sends reservation request to 1; 1 reads D2 and writes D3; 2 reads D3 and updates D2 status; 4 reads D3 and D4 and issues the bill and receipt to Guest; 5 reads all stores to give reports to Manager.

9.3 Borrowing a Book (DFD)

Assumptions: maximum of 3 books per member; if a book is unavailable the member may reserve it.

Step	Process	Reads / writes	Flow
1	Validate member	D1 Members	Member ID in; valid or invalid out
2	Check loan limit	D3 Loans	Current loans counted; reject if 3 already
3	Check book availability	D2 Books	Available or not
4a	Issue book (if available)	Update D2 copies; write D3 Loan	Issue slip to Member
4b	Reserve book (if unavailable)	Write D4 Reservations	Reservation confirmation to Member

Draw with Member as external entity, four data stores, and a decision expressed through separate labelled flows ('available' and 'not available') leaving process 3. DFDs show data flow, not control flow, so avoid flowchart-style diamonds.

9.4 Movie-Time Cinemas (use case and class diagram)

Assumptions: customers can view shows, book seats online or at the counter, and pay; each screen hosts shows with seat maps; staff manage schedules.

Element	Items
Actors	Customer, Box-office staff, Manager, Payment gateway
Use cases	View movies and showtimes, Select seats, Book ticket, Make payment, Cancel booking, Print ticket, Manage shows, Manage pricing, Generate reports
Classes (nouns)	Customer, Movie, Show, Screen, Seat, Booking, Ticket, Payment, Staff
Attributes (samples)	Movie (title, duration, language, rating); Show (showID, date, time); Seat (seatNo, type, price); Booking (bookingID, date, status); Payment (amount, mode, status)
Operations (samples)	Customer: search(), book(), cancel(); Booking: confirm(), cancel(), calculateTotal(); Show: checkAvailability(); Payment: process(), refund()
Relationships	Customer 1 makes 0..* Booking; Booking 1 has 1..* Ticket; Show 1 has 0..* Booking; Screen 1 hosts 0..* Show; Movie 1 is screened in 0..* Show; Screen 1 contains 1..* Seat (composition); Booking 1 has 1 Payment; Staff manages Show; Regular and Premium Customer inherit from Customer

DFD rules and method

- **Method:** draw the **context diagram** (one process, all external entities) first, then decompose into Level 1, Level 2 as needed.
- **Balancing:** inputs and outputs of a parent process must match the flows on its child diagram.
- **Numbering:** processes are numbered hierarchically (1, 2, 3, then 3.1, 3.2). Each process has a verb-noun name.
- **No entity-to-entity flows;** every flow must pass through a process.

- **Common errors:** data flowing directly between two stores or between an entity and a store; process with no input (miracle) or no output (black hole); unlabeled flows; control flow drawn as decisions; unbalanced levels; too many processes (more than about 7-9) on one diagram.

ER details

- **Cardinality:** 1:1 (one manager heads one department), 1:N (one department has many employees), M:N (students enrol in many courses, and vice versa).
- **Attribute types:** *key* (unique identifier, underlined); *composite* (splits into parts, e.g. Name into First and Last); *multivalued* (several values, e.g. Phone; double ellipse); *derived* (computed, e.g. Age from DOB; dashed ellipse); plus simple attributes.
- **Weak entity:** has no key of its own and depends on an owner entity (e.g. DEPENDENT of EMPLOYEE); double rectangle, with an identifying relationship (double diamond) and partial key.
- **Participation:** *total* (double line): every entity instance must take part; *partial* (single line): may or may not.

Class diagram method

- 1 **Identify classes from the nouns** in the problem statement; drop duplicates, vague terms and attributes.
- 2 **Add attributes** (data) and **operations** (behaviour, verbs) to each class.
- 3 **Connect with relationships** and add **multiplicity** (1, 0..1, 0..*, 1..*).
- 4 **Association:** a general link (Customer books Ticket). **Aggregation:** 'has-a', parts can exist independently (hollow diamond; Library has Books). **Composition:** strong 'part-of', parts die with the whole (filled diamond; Screen contains Seats). **Inheritance:** 'is-a' generalisation (hollow triangle; PremiumCustomer is a Customer).

Use case method

- **Identify actors:** who uses the system, and which external systems interact with it.
- **Identify use cases:** goals each actor wants to achieve; name them verb-noun.
- **Write a use case description:** name, actor, preconditions, main flow (numbered steps), alternative flows, postconditions.
- **<<include>>:** mandatory shared behaviour always executed (Borrow book includes Check availability).
- **<<extend>>:** optional or conditional behaviour (Pay fine extends Return book only when the book is late).

10. Data Dictionary

- **Purpose:** a central repository of definitions for every data flow, data store and data element used in the DFDs, so that everyone interprets the data the same way.
- **Relationship to DFD:** the DFD shows where data moves; the data dictionary defines exactly what the data is. Every named flow and store in the DFD must have an entry.

Symbol	Meaning
=	is composed of
+	and (sequence)
[]	either / or (selection)
()	optional
{ }	repetition (iteration)
* *	comment

```

Borrower-Details = Name + Address + (Phone)
Name = First-Name + (Middle-Name) + Last-Name
Member-Type = [ Student | Faculty | Staff ]
Loan-Record = Member-ID + {Book-ID + Due-Date}3
Book-ID = *unique ISBN-based identifier*

```

11. SRS: Small Gaps

Advantages of SRS standards

- Provides a **basis for agreement** between customer and developer on what is to be built.
- **Reduces development effort:** a thorough review forces early detection of omissions and inconsistencies.
- Provides a **basis for estimation and validation:** size, cost, schedule and test cases come from it.
- Makes **transfer** of the software to new users and machines easier.
- Provides a **basis for enhancement** and later maintenance.

Nature of SRS

- **Functional requirements:** what the system must do (services, inputs, outputs, processing).
- **Non-functional requirements:** how well it does it (performance, reliability, security, usability, portability, constraints).
- The SRS says **what** the system should do, not **how** it will be done; design decisions belong to the design phase.

UNIT II

1. Deliverables and Milestones

Term	Definition	Example
Measure	A quantitative indication of an attribute of a product or process	20 errors found in a module
Measurement	The act of determining a measure	Counting errors in module A
Metric	A quantitative measure of the degree to which a system, component or process has an attribute	Errors per KLOC
Deliverable	A tangible output handed over to the customer at the end of a phase or project	SRS, source code, user manual, test report
Milestone	A checkpoint that marks completion of a phase or activity, used to track progress	'SRS approved', 'Design review complete'

Deliverables are what the customer receives; milestones are internal or shared progress markers. A milestone is usually reached when a deliverable is completed and reviewed.

2. Data Structure Metrics

Need: the complexity of a program depends on the data it handles as well as on its logic. Data structure metrics measure how data is used and shared, to predict complexity, testing effort and maintenance effort.

- **Amount of data:** how much data the program handles (number of variables and data items, e.g. Halstead's n_2 and N_2 operand counts).
- **Usage of data within a module:** how the variables are used in the module. Measured with **live variables**, **variable span** and **average span size**.
- **Sharing of data among modules:** data shared through global variables and parameters; more sharing means stronger coupling, more complexity and more risk from change.

Live variables (LV)

A variable is **live** from its first reference to its last reference within a module. The number of live variables at a statement gives that statement's LV; the module's **average LV** = total LV over statements divided by number of statements. More live variables means more to keep track of and higher complexity.

Span (SP)

The **span** of a variable is the number of statements between two successive references to it. **Average span size** = total of all spans divided by the number of spans. A large span means the variable is used far from where it is last used, so it is harder to reason about.

Short example

```
1 read(a)
2 read(b)
3 c = a + b
4 d = c * 2
5 print(d)
```

Statement	Live variables	LV
1	a	1
2	a, b	2
3	a, b, c	3
4	c, d	2
5	d	1

Total LV = 9 over 5 statements, so **average LV = 1.8**. Spans: a (referenced at 1 and 3) has span 1; b (2 and 3), c (3 and 4), d (4 and 5) each have span 0. Total = 1 over 4 spans, so **average span size = 0.25**.

3. Software Project Planning Activities

- **Size estimation:** estimate size of the product (LOC or function points).
- **Cost estimation:** derive effort and money from size (COCOMO, function points).
- **Schedule estimation:** determine duration, task order and dependencies (Gantt, PERT, CPM).
- **Staffing:** decide how many people, with what skills, and when.
- **Resource planning:** hardware, software tools, environments, training and budget.
- **Risk management planning:** identify, assess and plan for risks.
- **Quality assurance planning:** standards, reviews, testing and configuration control.
- **Monitoring and control:** track progress against the plan and take corrective action.

4. Halstead: Missing Formulas and Problem Method

Formula sheet

Notation: n_1 = distinct operators; n_2 = distinct operands; N_1, N_2 = total operator and operand occurrences; n_2^* = number of distinct input/output parameters.

Quantity	Formula
Vocabulary	$n = n_1 + n_2$
Length	$N = N_1 + N_2$
Estimated length	$\hat{N} = n_1 \cdot \log_2 n_1 + n_2 \cdot \log_2 n_2$
Volume	$V = N \cdot \log_2 n$
Potential volume	$V^* = (2 + n_2^*) \cdot \log_2 (2 + n_2^*)$
Program level	$L = V^* / V$
Estimated level	$\hat{L} = (2 / n_1) \cdot (n_2 / N_2)$, which equals $1/D$
Difficulty	$D = 1/L = (n_1/2) \cdot (N_2/n_2)$
Effort	$E = V / L = D \cdot V$
Language level	$\lambda = L \cdot V^* (= L^2 \cdot V)$
Time	$T = E / \text{beta}$, with $\text{beta} = 18$, so $T = E/18$ seconds

Worked numerical: $n_1 = 40, n_2 = 60, N_1 = 16, N_2 = 21$

Assumptions: the question gives no n_2^* , so compute L from the estimator and take $V^* = L \times V$. Also note that $N_1 < n_1$ is impossible in a real program (each distinct operator must appear at least once), but solve it as given.

- 1 $n = 40 + 60 = \mathbf{100}$; $N = 16 + 21 = \mathbf{37}$.
- 2 $\hat{N} = 40 \cdot \log_2 40 + 60 \cdot \log_2 60 = 40(5.322) + 60(5.907) = 212.9 + 354.4 = \mathbf{567.3}$.
- 3 $V = 37 \times \log_2 100 = 37 \times 6.644 = \mathbf{245.8}$.
- 4 $\hat{L} = (2/40)(60/21) = 0.05 \times 2.857 = \mathbf{0.1429}$. $D = 1/L = 7$.
- 5 $V^* = L \times V = 0.1429 \times 245.8 = \mathbf{35.1}$ (assumed).
- 6 $E = V / L = 245.8 / 0.1429 = \mathbf{1720.8}$.
- 7 $T = E / 18 = 1720.8 / 18 = \mathbf{95.6 \text{ seconds}}$.
- 8 $\lambda = L \cdot V^* = 0.1429 \times 35.1 = \mathbf{5.02}$.

5. Function Points: Constants the PPT Leaves Out

Weight table

Component	Low (simple)	Average	High (complex)
External Inputs (EI)	3	4	6
External Outputs (EO)	4	5	7
External Inquiries (EQ)	3	4	6

Component	Low (simple)	Average	High (complex)
Internal Logical Files (ILF)	7	10	15
External Interface Files (EIF)	5	7	10

The 14 adjustment (complexity) factors

#	Factor	#	Factor
1	Reliable backup and recovery	8	Master files updated online
2	Data communication	9	Complex inputs, outputs, files or inquiries
3	Distributed processing	10	Complex internal processing
4	Performance critical	11	Code designed to be reusable
5	Existing heavily used operational environment	12	Conversion and installation in the design
6	On-line data entry	13	Multiple installations
7	Input transaction over multiple screens	14	Application designed to facilitate change

The 0-5 rating scale

0	1	2	3	4	5
No influence	Incidental	Moderate	Average	Significant	Essential

Exam wording 'average', 'moderate' and 'essential' maps onto ratings 3, 2 and 5.

Formulas and one full computation

Count Total = sum of (count x weight) over all 5 components
CAF = $0.65 + 0.01 \times (\text{sum of } F_i)$
FP = Count Total x CAF

Example: EI = 5, EO = 4, EQ = 3, ILF = 2, EIF = 1, all of average complexity; all 14 factors rated average (3).

Component	Count	Weight (avg)	Product
EI	5	4	20
EO	4	5	20
EQ	3	4	12
ILF	2	10	20
EIF	1	7	7
Count Total			79

Sum of $F_i = 14 \times 3 = 42$; CAF = $0.65 + 0.42 = 1.07$; FP = $79 \times 1.07 = 84.53$.

6. COCOMO: Gaps

Basic COCOMO (standard table, from the PPT)

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

$E = a \times (\text{KLOC})^b$ person-months
 $D = c \times E^d$ months

Intermediate COCOMO constants (differ from Basic)

Mode	a	b
Organic	3.2	1.05

Mode	a	b
Semi-detached	3.0	1.12
Embedded	2.8	1.20

$E = a \times (\text{KLOC})^b \times \text{EAF}$ EAF = product of the 15 cost-driver multipliers
 $D = 2.5 \times E^d$ (time formula is unchanged)

Cost driver multipliers

A driver rated **Nominal** has multiplier **1.00**. Ratings above or below nominal raise or lower the effort. Examples:

Driver	Range (Very Low to Very High or Extra High)	Meaning
RELY (required reliability)	0.75 to 1.40	Higher reliability need means more effort
CPLX (product complexity)	0.70 to 1.65	More complex product means more effort
ACAP (analyst capability)	1.46 to 0.71	Better analysts mean less effort (inverse)
SCED (required schedule)	1.23 to 1.00	Compressed schedule raises effort

Detailed COCOMO

- It uses the same equations as Intermediate but the cost-driver multipliers are **phase-sensitive**: each driver has a different multiplier for each phase (plan and requirements, product design, detailed design, coding and unit test, integration and test).
- The project is decomposed in a **three-level hierarchy: module, subsystem and system**. Effort is estimated for modules, then rolled up to subsystems and the whole system, giving more accurate estimates for large projects.
- The product is more accurate but needs far more data and effort to apply.

Model	Inputs	Accuracy	Use
Basic	KLOC and mode	Rough	Early quick estimate
Intermediate	KLOC, mode and 15 cost drivers	Better	Most common
Detailed	Intermediate plus phase-wise drivers and hierarchy	Best	Large, well-understood projects

Derived quantities and worked example

Average staff = E / D Productivity = KLOC / E (KLOC per person-month)

Example: 32 KLOC organic product, Intermediate COCOMO, EAF = 1.15. $E = 3.2 \times 32^{1.05} \times 1.15 = 3.2 \times 38.05 \times 1.15 = 140.0 \text{ PM}$. $D = 2.5 \times 140^{0.38} = 2.5 \times 6.54 = 16.4 \text{ months}$. Average staff = $140 / 16.4 = 8.5 \text{ persons}$.
 Productivity = $32 / 140 = 0.229 \text{ KLOC/PM}$ (about 229 LOC/PM).

7. Walston-Felix and SEL Models

Static single-variable models estimate effort and duration from a single variable, size (L, in KLOC), using the form **Effort = $a \times L^b$** . They are simpler alternatives to COCOMO and do not use cost drivers; the constants come from the historical data of a particular environment.

	Walston-Felix	SEL (Software Engineering Laboratory)
Effort E (person-months)	$E = 5.2 \times L^{0.91}$	$E = 1.4 \times L^{0.93}$
Duration D (months)	$D = 4.1 \times L^{0.36}$	$D = 4.6 \times L^{0.26}$
Average staff	E / D	E / D
Productivity (LOC per person-month)	$L \times 1000 / E$	$L \times 1000 / E$

Solving for size when effort is given

- Convert effort to person-months (person-years $\times 12$).

- 2 Invert the effort equation: $L = (E / a)^{1/b}$.
- 3 Compute duration D from the duration equation.
- 4 Productivity = L (in LOC) / E; average manning = E / D.

Worked example: E = 8 person-years

$E = 8 \times 12 = 96$ person-months.

Quantity	Walston-Felix	SEL
Size L	$(96/5.2)^{1/0.91} = 18.46^{1.099} = \mathbf{24.6 \text{ KLOC}}$	$(96/1.4)^{1/0.93} = 68.57^{1.075} = \mathbf{94.3 \text{ KLOC}}$
Duration D	$4.1 \times 24.6^{0.36} = 4.1 \times 3.17 = \mathbf{13.0 \text{ months}}$	$4.6 \times 94.3^{0.26} = 4.6 \times 3.26 = \mathbf{15.0 \text{ months}}$
Productivity	$24,600 / 96 = \mathbf{257 \text{ LOC/PM}}$	$94,300 / 96 = \mathbf{982 \text{ LOC/PM}}$
Average manning	$96 / 13.0 = \mathbf{7.4 \text{ persons}}$	$96 / 15.0 = \mathbf{6.4 \text{ persons}}$

Comparison

- For the same effort, SEL implies a much **larger** product (its 'a' is smaller), meaning higher productivity, than Walston-Felix. This reflects the environments the data came from.
- Both are static single-variable models with no cost drivers. Neither adapts to project attributes, unlike COCOMO, so they should be calibrated to local data.